



Python for Data Science - 2305CS303

Lab - 12

Roll No. : 122

Name : Yatharth Sanghavi

1. Concatenate DataFrames Row-wise

Create two DataFrames containing student details from two different groups. Use `pd.concat()` with `axis=0` to concatenate the DataFrames row-wise. Display the original DataFrames and the resulting DataFrame.

```
import pandas as pd

group1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
})
group2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"],
    "City": ["Ahmedabad", "Rajkot", "Surat"]
})

print("Group 1")
print(group1)
print("\nGroup 2")
print(group2)

result = pd.concat([group1, group2], axis=0)
print("\nConcatenated (row-wise)")
print(result)
```

```
Group 1
  RollNo  Name    City
0      1  Amit  Rajkot
1      2  Riya   Surat
2      3  Karan Vadodara
```

```
Group 2
  RollNo  Name    City
0      4  Neha Ahmedabad
1      5   Raj  Rajkot
2      6 Pooja   Surat
```

```
Concatenated (row-wise)
  RollNo  Name    City
0      1  Amit  Rajkot
1      2  Riya   Surat
2      3  Karan Vadodara
0      4  Neha Ahmedabad
1      5   Raj  Rajkot
2      6 Pooja   Surat
```

2. Concatenate DataFrames Column-wise

Create two DataFrames containing student personal details and academic details with the same index. Use `pd.concat()` with `axis=1` to combine the DataFrames column-wise. Display the resulting DataFrame.

```
personal = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
}, index=[1, 2, 3])
academic = pd.DataFrame({
    "Marks": [78, 85, 72],
    "Grade": ["B", "A", "C"]
}, index=[1, 2, 3])

result = pd.concat([personal, academic], axis=1)
print(result)
```

	Name	City	Marks	Grade
1	Amit	Rajkot	78	B
2	Riya	Surat	85	A
3	Karan	Vadodara	72	C

3. Concatenate DataFrames with Different Columns

Create two DataFrames containing student records with some common and some different columns. Use `pd.concat()` with `axis=0` and observe how pandas handles columns that are not present in both DataFrames.

```
df1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
})
df2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"],
    "Marks": [80, 75, 90]
})

result = pd.concat([df1, df2], axis=0)
print(result)
# Columns not present in both DataFrames (City, Marks) are filled with NaN
```

	RollNo	Name	City	Marks
0	1	Amit	Rajkot	NaN
1	2	Riya	Surat	NaN
2	3	Karan	Vadodara	NaN
0	4	Neha	NaN	80.0
1	5	Raj	NaN	75.0
2	6	Pooja	NaN	90.0

4. Concatenate DataFrames and Reset the Index

Create two DataFrames representing student records from two classes, each having its own index. Concatenate them row-wise using `pd.concat()` and use `ignore_index=True` to generate a new continuous index.

```
class1 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["Amit", "Riya", "Karan"]
}, index=[0, 1, 2])
class2 = pd.DataFrame({
    "RollNo": [4, 5, 6],
    "Name": ["Neha", "Raj", "Pooja"]
}, index=[0, 1, 2])

print("Without ignore_index")
print(pd.concat([class1, class2], axis=0))

print("\nWith ignore_index=True")
print(pd.concat([class1, class2], axis=0, ignore_index=True))
```

	RollNo	Name
0	1	Amit
1	2	Riya
2	3	Karan
0	4	Neha

```

1      5    Raj
2      6   Pooja

With ignore_index=True
  RollNo  Name
0       1   Amit
1       2   Riya
2       3   Karan
3       4   Neha
4       5    Raj
5       6   Pooja

```

5. Concatenate Multiple DataFrames

Create three DataFrames containing student records from three semesters. Use `pd.concat()` to combine all three DataFrames row-wise. Display the final DataFrame and its shape.

```

sem1 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [1, 1]})
sem2 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [2, 2]})
sem3 = pd.DataFrame({"RollNo": [1, 2], "Name": ["Amit", "Riya"], "Semester": [3, 3]})

result = pd.concat([sem1, sem2, sem3], axis=0, ignore_index=True)
print(result)
print("Shape:", result.shape)

```

```

  RollNo  Name  Semester
0       1   Amit         1
1       2   Riya         1
2       1   Amit         2
3       2   Riya         2
4       1   Amit         3
5       2   Riya         3
Shape: (6, 3)

```

6. Join Two DataFrames using Index

Create two DataFrames containing student information and marks, with `RollNo` as the index. Use the `join()` method to combine the DataFrames based on their index. Display the resulting DataFrame.

```

student_info = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan"],
    "City": ["Rajkot", "Surat", "Vadodara"]
}, index=pd.Index([101, 102, 103], name="RollNo"))

marks = pd.DataFrame({
    "Python": [78, 85, 72],
    "DBMS": [80, 90, 65]
}, index=pd.Index([101, 102, 103], name="RollNo"))

result = student_info.join(marks)
print(result)

```

```

      Name      City  Python  DBMS
RollNo
101   Amit   Rajkot      78     80
102   Riya    Surat      85     90
103   Karan  Vadodara      72     65

```

7. Join DataFrames with Different Index Values

Create two DataFrames containing student details and attendance information with partially matching indexes. Demonstrate `join()` using the default left join and then using `how='inner'`. Compare the results.

```

details = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan", "Neha"],
    "City": ["Rajkot", "Surat", "Vadodara", "Ahmedabad"]
}, index=pd.Index([101, 102, 103, 104], name="RollNo"))

attendance = pd.DataFrame({
    "Attendance": [90, 85, 70]
}, index=pd.Index([101, 102, 105], name="RollNo"))

print("Left join (default)")
print(details.join(attendance))

print("\nInner join")
print(details.join(attendance, how="inner"))

```

```
# Left join keeps all rows of the left DataFrame (NaN where no match),
# inner join keeps only the RollNo values present in both.
```

```
Left join (default)
      Name      City  Attendance
RollNo
101    Amit    Rajkot        90.0
102    Riya    Surat         85.0
103    Karan  Vadodara         NaN
104    Neha  Ahmedabad         NaN
```

```
Inner join
      Name      City  Attendance
RollNo
101    Amit  Rajkot         90
102    Riya  Surat         85
```

8. Merge DataFrames using a Common Column

Create two DataFrames containing student details and examination marks. Both DataFrames should contain a common column named `RollNo`. Use `pd.merge()` to combine them based on `RollNo` and display the result.

```
student = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Name": ["Amit", "Riya", "Karan", "Neha"]
})
exam = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Marks": [78, 85, 72, 90]
})

result = pd.merge(student, exam, on="RollNo")
print(result)
```

```
RollNo  Name  Marks
0      1  Amit    78
1      2  Riya    85
2      3  Karan    72
3      4  Neha    90
```

9. Merge DataFrames using Different Join Types

Create two DataFrames containing student records from two different sources with some common and some non-common `RollNo` values. Use `pd.merge()` to demonstrate `inner`, `left`, `right`, and `outer` joins. Display and compare all four results.

```
source1 = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Name": ["Amit", "Riya", "Karan", "Neha"]
})
source2 = pd.DataFrame({
    "RollNo": [3, 4, 5, 6],
    "Marks": [72, 90, 80, 65]
})

print("Inner join")
print(pd.merge(source1, source2, on="RollNo", how="inner"))

print("\nLeft join")
print(pd.merge(source1, source2, on="RollNo", how="left"))

print("\nRight join")
print(pd.merge(source1, source2, on="RollNo", how="right"))

print("\nOuter join")
print(pd.merge(source1, source2, on="RollNo", how="outer"))
```

```
Inner join
RollNo  Name  Marks
0      3  Karan    72
1      4  Neha    90
```

```
Left join
RollNo  Name  Marks
0      1  Amit   NaN
1      2  Riya   NaN
2      3  Karan  72.0
3      4  Neha  90.0
```

```
Right join
RollNo  Name  Marks
0      3  Karan    72
```

1	4	Neha	90
2	5	NaN	80
3	6	NaN	65

Outer join

	RollNo	Name	Marks
0	1	Amit	NaN
1	2	Riya	NaN
2	3	Karan	72.0
3	4	Neha	90.0
4	5	NaN	80.0
5	6	NaN	65.0

10. Combine Student Academic Data using Concat, Join and Merge

Create three DataFrames containing student personal information, semester marks, and attendance information. Use `pd.concat()` to combine two suitable DataFrames, `join()` to combine data using an index, and `pd.merge()` to combine data using a common column. Display the final combined student dataset.

```
# Personal information of students (two batches)
personal_a = pd.DataFrame({
    "RollNo": [1, 2],
    "Name": ["Amit", "Riya"],
    "City": ["Rajkot", "Surat"]
})
personal_b = pd.DataFrame({
    "RollNo": [3, 4],
    "Name": ["Karan", "Neha"],
    "City": ["Vadodara", "Ahmedabad"]
})

# Semester marks (RollNo as index)
marks = pd.DataFrame({
    "Sem1": [70, 80, 65, 90],
    "Sem2": [75, 85, 60, 88]
}, index=pd.Index([1, 2, 3, 4], name="RollNo"))

# Attendance (RollNo as a column)
attendance = pd.DataFrame({
    "RollNo": [1, 2, 3, 4],
    "Attendance": [92, 88, 75, 95]
})

# Step 1: concat -> combine the two personal DataFrames
personal = pd.concat([personal_a, personal_b], ignore_index=True)

# Step 2: join -> add marks using the index
personal_indexed = personal.set_index("RollNo")
with_marks = personal_indexed.join(marks)
with_marks = with_marks.reset_index()

# Step 3: merge -> add attendance using the common column RollNo
final = pd.merge(with_marks, attendance, on="RollNo")

print("Final combined student dataset")
print(final)
```

Final combined student dataset

	RollNo	Name	City	Sem1	Sem2	Attendance
0	1	Amit	Rajkot	70	75	92
1	2	Riya	Surat	80	85	88
2	3	Karan	Vadodara	65	60	75
3	4	Neha	Ahmedabad	90	88	95

